

天津大学

《操作系统设计综合实践》课程报告

基于 xv6-RISC-V 的静态优先级调度与

Aging 防饥饿机制



学	院	软件学院
专	业	软件工程
组	号	第 35 组
小组成员	赖弈铭	3022002026
	刘文睿	3023001172
	吴雨桐	3022001567
	党正清	3023244304
	袁晟棋	3023244399

2026 年 9 月 21 日

目 录

第一章	项目概述	1
1.1	项目背景	1
1.2	小组分工	1
1.3	AI 使用情况	1
第二章	设计与实现	2
2.1	总体设计	2
2.2	进程优先级数据	3
2.3	静态优先级调度	5
2.4	Aging 防饥饿机制	6
2.5	系统调用与用户命令	7
2.6	测试与验证	10
第三章	问题分析与解决	12
3.1	QEMU 启动时动态库符号不匹配	12
3.2	fork 继承优先级时缺少锁保护	12
3.3	xPack QEMU 版本号解析失败	12
3.4	Aging 与手动调优的状态衔接	12
第四章	总结与展望	13

第一章 项目概述

1.1 项目背景

本项目是“操作系统设计综合实践”课程的小组课题，开发基线为 xv6-RISC-V。xv6 原有调度器采用轮转方式遍历进程表，对所有可运行进程一视同仁。这种实现结构清楚，适合教学，但无法表达任务之间的优先级差异。项目在保留 xv6 进程状态转换和锁机制的前提下，引入静态优先级调度，并使用 Aging 缓解低优先级进程长期等待的问题。

项目规定优先级取值范围为 0 至 31，数值越小表示优先级越高，默认值为 10。实现范围包括进程优先级字段、fork 继承、静态优先级调度、同级轮转、Aging、优先级系统调用和用户命令。代码仓库地址为：github.com/Aphrodite-deepblue/os-design-practice。

1.2 小组分工

小组按内核数据、调度策略、Aging、系统调用和测试五个模块分工。每项任务都对应明确的代码区域和 Git 提交，分工见表1-1。

表 1-1 第 35 组成员及分工

姓名	学号	GitHub 账号	主要任务
赖弈铭	3022002026	Aphrodite-deepblue	Aging、等待计数与防饥饿逻辑
刘文睿	3023001172	milabo0718	静态优先级调度与同优先级轮转
吴雨桐	3022001567	Curtai123	测试设计、集成验证与结果整理
党正清	3023244304	dzq180	优先级系统调用与 nice 命令
袁晟棋	3023244399	yuan-sq	进程优先级数据、fork 继承与集成

1.3 AI 使用情况

第二章 设计与实现

2.1 总体设计

本项目没有替换 xv6 的进程管理框架，而是在原有轮转调度路径上增加一层优先级选择。核心状态仍保存在 struct proc 中，进程状态仍遵循 UNUSED、USED、SLEEPING、RUNNABLE、RUNNING 和 ZOMBIE 的既有转换规则。新增逻辑只在持有相应 $p \rightarrow \text{lock}$ 时读取或修改调度字段，从而使实现能够适应 xv6 的多 CPU 配置。

系统的主路径如图2-1所示。用户命令 nice 负责输入检查和结果显示；setpriority、getpriority 将用户请求转入内核；进程控制块保存 priority 与 wait_ticks；调度器依据这两个字段完成最高优先级选择、同级轮转和 Aging。Aging 不是独立调度器，而是调度扫描中的一次状态更新，因此不会把本项目扩展为 MLFQ 或其他动态优先级模型。

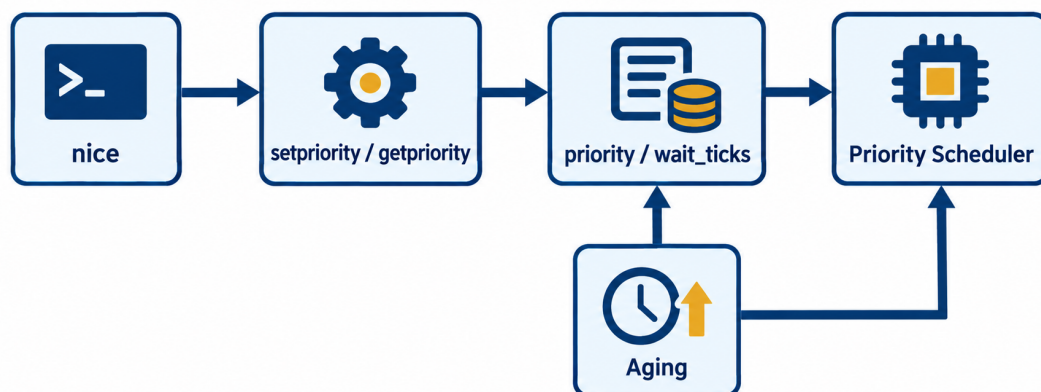


图 2-1 优先级调度功能的模块关系

五个模块之间的边界较为清楚：用户态只通过系统调用修改可见属性，系统调用层只负责参数传递，内核进程函数负责校验和加锁，调度器负责选择，Aging 只处理长期等待。这样的拆分既便于五名成员分别开发，也减少了多处代码各自解释优先级语义的风险。

本项目统一规定优先级范围为 0 至 31，数值越小表示优先级越高，默认值为 10。若可运行进程集合为 R ，调度器首先计算

$$p_{\text{best}} = \min_{p \in R} p.\text{priority},$$

再从满足 `priority == p_best` 的进程中按轮转起点选择一个。这个过程把“不同优先级之间的先后关系”与“相同优先级之间的公平性”分开处理，便于检查每一步是否符合预期。

2.2 进程优先级数据

2.2.1 字段与取值约定

`kernel/proc.h` 集中定义优先级边界、默认值和 Aging 阈值，并在进程控制块中增加两个字段。常量集中管理后，调度器、系统调用和进程创建代码都调用同一个 `priority_valid()`，避免不同模块采用不一致的范围判断。关键定义见代码2.1。

```

1 #define PRIORITY_MIN      0
2 #define PRIORITY_MAX      31
3 #define PRIORITY_DEFAULT 10
4 #define AGING_INTERVAL    50
5
6 static inline int
7 priority_valid(int priority)
8 {
9     return priority >= PRIORITY_MIN &&
10         priority <= PRIORITY_MAX;
11 }
12
13 struct proc {
14     struct spinlock lock;
15     enum procstate state;
16     int pid;
17     int priority;
18     uint wait_ticks;
19     /* other xv6 process fields */
20 };

```

Listing 2.1 进程优先级常量与调度字段 (`kernel/proc.h`)

`priority` 表示进程当前用于调度比较的值。它在通常情况下保持不变，只有用户显式设置或 Aging 生效时才发生改变。`wait_ticks` 不是系统时钟的绝对 tick，而是进程保持 `RUNNABLE` 并被调度器扫描的等待轮次。采用扫描轮次可以把实现限制在 `proc.c` 内，不必修改时钟中断路径；其代价是该数值更适合表达“等

待程度”，不应被解释为精确运行时间。

2.2.2 初始化、继承与回收

新增字段必须覆盖完整生命周期。procinit() 和 allocproc() 都把优先级设为 10、等待计数设为 0；前者建立进程表的初始不变量，后者保证复用槽位时再次初始化。freeproc() 在进程槽位回到 UNUSED 前恢复默认值，防止残留状态泄漏给后续进程。各阶段的处理见表2-1。

表 2-1 调度字段在进程生命周期中的处理

位置	priority	wait_ticks
初始化与分配	设为默认值 10	清零
fork	继承父进程的加锁快照	清零
获得 CPU	保持当前值	清零
手动设置优先级	写入校验后的新值	清零
释放进程槽位	恢复默认值 10	清零

fork 继承不能直接读取父进程字段。主分支实现先获取父进程锁，保存优先级快照，再继续复制地址空间。子进程的 np->lock 由 allocproc() 保持，因此可直接写入继承值。若快照异常，则回退到默认优先级，保证子进程进入 RUNNABLE 前一定具有合法值。关键代码见代码2.2。

```

1  acquire(&p->lock);
2  parent_priority = p->priority;
3  release(&p->lock);
4
5  /* Copy user memory and trapframe. */
6
7  np->priority = priority_valid(parent_priority)
8                ? parent_priority
9                : PRIORITY_DEFAULT;
10 np->wait_ticks = 0;

```

Listing 2.2 fork 中的优先级继承 (kernel/proc.c)

父进程锁只用于取得快照，没有跨越耗时的地址空间复制过程。这样既保护共享字段，也避免无谓延长临界区。子进程等待计数从 0 开始，因为它尚未参与过调度竞争，不能继承父进程此前积累的等待历史。

2.3 静态优先级调度

2.3.1 两轮扫描策略

调度器采用两轮扫描。第一轮逐项获取进程锁，更新 Aging 状态，并在全部 RUNNABLE 进程中求出最小优先级。第二轮从当前 CPU 保存的 next_index 开始环形遍历，选择一个优先级等于最优值的进程。选中后把状态改为 RUNNING，将等待计数清零，再调用 swtch() 完成上下文切换。

代码2.3展示了调度核心。第一轮把所有仍在等待的可运行进程计数加一，随后确定 best_priority；第二轮再次检查状态与优先级。这次复查是必要的，因为在多 CPU 情况下，另一个 CPU 可能在两轮之间调度或修改候选进程。

```
1  int best_priority = PRIORITY_MAX + 1;
2  int found = 0;
3
4  for (p = proc; p < &proc[NPROC]; p++) {
5      acquire(&p->lock);
6      apply_aging(p);
7      if (p->state == RUNNABLE) {
8          p->wait_ticks++;
9          if (p->priority < best_priority) {
10             best_priority = p->priority;
11             found = 1;
12         }
13     }
14     release(&p->lock);
15 }
16
17 if (found) {
18     int selected = 0;
19     for (int offset = 0; offset < NPROC; offset++) {
20         int index = (next_index + offset) % NPROC;
21         p = &proc[index];
22         acquire(&p->lock);
23         if (p->state == RUNNABLE &&
24             p->priority == best_priority) {
25             p->state = RUNNING;
26             c->proc = p;
```

```

27     p->wait_ticks = 0;
28     swtch(&c->context, &p->context);
29     c->proc = 0;
30     next_index = (index + 1) % NPROC;
31     selected = 1;
32     release(&p->lock);
33     break;
34 }
35 release(&p->lock);
36 }
37 if (selected)
38     continue;
39 }

```

Listing 2.3 静态优先级调度与同级轮转核心(kernel/proc.c)

这里没有在进程表外维护复杂的优先级队列。每轮扫描的时间复杂度仍为 $O(NPROC)$ ，与教学版 xv6 遍历进程表的思路一致。虽然两轮扫描增加了一次遍历，但它把“找出最高优先级”和“在同级中选择”拆开，代码更容易核查，也避免引入额外队列的一致性维护成本。

2.3.2 同优先级公平与多 CPU 处理

如果第二轮总从 `proc[0]` 开始，同一优先级下靠前的进程会反复先被发现。为此，每个 CPU 在进入 `scheduler()` 时建立局部变量 `next_index`，初值为 `cpuid() % NPROC`。进程运行结束并返回调度器后，起点移动到该进程的下一个槽位。由此，同一最优优先级内的进程按环形顺序获得机会。

该公平机制是“基本公平”，并不承诺严格的全局轮转。每个 CPU 拥有独立起点，且进程状态会并发变化，因此实际顺序可能受唤醒、退出和其他 CPU 选择影响。但每个候选都必须在持锁后再次满足 `RUNNABLE` 和最优优先级条件，不会因为使用轮转起点而绕过 xv6 的锁和状态规则。

若第一轮找到可运行进程，但第二轮没有成功选中，调度器不会立即执行 `wfi`。这种情况通常意味着候选在两轮间被其他 CPU 取走或改变状态；当前 CPU 会重新开始下一轮选择。只有第一轮确认没有任何可运行进程时，才进入等待中断状态。

2.4 Aging 防饥饿机制

静态优先级能够表达任务先后，但持续出现的高优先级任务可能使低优先级进程长期得不到 CPU。Aging 的目标是在不改变静态优先级框架的前提下，为持

续等待的进程逐级提升调度机会。实现使用命名常量 `AGING_INTERVAL=50`，避免在调度器中散落难以解释的数字。

```

1  static void
2  apply_aging(struct proc *p)
3  {
4      if (p->state == RUNNABLE &&
5          p->wait_ticks >= AGING_INTERVAL &&
6          priority_valid(p->priority) &&
7          p->priority > PRIORITY_MIN) {
8          p->priority -= 1;
9          p->wait_ticks = 0;
10     }
11 }
```

Listing 2.4 Aging 状态更新 (kernel/proc.c)

代码2.4由调度器在持有 `p->lock` 时调用，只有 `RUNNABLE` 进程参与累计和提升。达到阈值后，优先级数值减一，即提升一级，同时清零等待计数。条件 `p->priority > PRIORITY_MIN` 保证结果不会越过合法上界 0；`priority_valid()` 则阻止异常字段继续参与提升。

等待计数在三个位置重置。进程真正被分派时清零，表示此前等待已经结束；用户通过系统调用设置新优先级时清零，使新策略从完整周期开始生效；创建、fork 和回收时也清零，避免继承无关历史。睡眠进程不累计，因为它没有处于可调度但未被选择的状态。

Aging 每次只提升一级，并在提升后重新等待一个完整阈值。设进程初始优先级为 p_0 ，连续等待并经历 k 次提升后，其优先级满足

$$p_k = \max(\text{PRIORITY_MIN}, p_0 - k).$$

因此，低优先级进程不会一步跳到最高优先级，也不会越界。该规则保留了静态优先级的主要排序，同时为长期等待者提供渐进式补偿。

2.5 系统调用与用户命令

2.5.1 内核接口与调用链

项目增加 `setpriority(pid, priority)` 和 `getpriority(pid)` 两个系统调用，编号分别为 23 和 24。编号没有覆盖原有调用。完整链路包括 `kernel/syscall.h` 的编号、`kernel/syscall.c` 的分发表、`kernel/sysproc.c` 的参数入口、`kernel/proc.c` 的进程表操作，以及 `user/user.h` 和 `user/usys.pl` 的用户态声明与调用桩。

表 2-2 优先级系统调用的分层实现

文件	作用
kernel/syscall.h	分配 23、24 两个系统调用编号
kernel/syscall.c	将编号映射到内核入口函数
kernel/sysproc.c	使用 argint() 读取 PID 与优先级参数
kernel/proc.c	校验参数、加锁遍历进程表并读写字段
user/user.h、user/ usys.pl	声明用户接口并生成 ecall 调用桩

真正的状态修改集中在 `ksetpriority()` 中。函数先拒绝非正 PID 和非法优先级，再逐项加锁查找目标进程。命中后同时写入优先级并清零 `wait_ticks`。查询函数采用相同的加锁遍历方式，先把字段复制到局部变量，再释放锁并返回，见代码2.5。

```

1  int
2  ksetpriority(int pid, int priority)
3  {
4      struct proc *p;
5      if (pid <= 0 || !priority_valid(priority))
6          return -1;
7
8      for (p = proc; p < &proc[NPROC]; p++) {
9          acquire(&p->lock);
10         if (p->state != UNUSED && p->pid == pid) {
11             p->priority = priority;
12             p->wait_ticks = 0;
13             release(&p->lock);
14             return 0;
15         }
16         release(&p->lock);
17     }
18     return -1;
19 }
20
21 int

```

```
22 kgetpriority(int pid)
23 {
24     struct proc *p;
25     int priority;
26     if (pid <= 0)
27         return -1;
28
29     for (p = proc; p < &proc[NPROC]; p++) {
30         acquire(&p->lock);
31         if (p->state != UNUSED && p->pid == pid) {
32             priority = p->priority;
33             release(&p->lock);
34             return priority;
35         }
36         release(&p->lock);
37     }
38     return -1;
39 }
```

Listing 2.5 内核优先级读写接口 (kernel/proc.c)

2.5.2 用户态 nice 命令

用户程序提供查询和设置两种形式: nice pid 查询指定进程, nice pid priority 设置新优先级。程序不直接使用宽松的 atoi(), 而是逐字符检查数字、拒绝空字符串与非数字字符, 并在十进制累加前检查 32 位有符号整数溢出。PID 为 0 会在用户态直接拒绝, 超出 0 至 31 的优先级则由统一的内核校验返回失败。

```
1 if (argc == 2) {
2     priority = getpriority(pid);
3     if (priority < 0) {
4         fprintf(2, "nice: process %d not found\n", pid);
5         exit(1);
6     }
7     printf("pid %d priority = %d\n", pid, priority);
8     exit(0);
9 }
10
```

```

11 if (parse_nonnegative_int(argv[2], &priority) < 0) {
12     fprintf(2, "nice: invalid priority '%s'\n", argv[2]);
13     exit(1);
14 }
15
16 if (setpriority(pid, priority) < 0) {
17     fprintf(2, "nice: failed to set pid %d priority to %d\n",
18             pid, priority);
19     exit(1);
20 }

```

Listing 2.6 nice 命令的查询与设置分支 (user/nice.c)

Makefile 将 `_nice` 加入 `UPROGS`，因此构建文件系统镜像时会自动安装该命令。用户输入检查、系统调用分层和内核范围校验共同构成完整接口：用户态负责给出明确错误信息，内核则始终保留最终校验权，避免其他用户程序绕过 `nice` 写入非法值。

2.6 测试与验证

第五阶段的测试程序已随主分支合并，测试入口为 `tests/run_scheduler_tests.py`，并在临时副本中分别构建单 CPU 与 3 CPU 的 `xv6` 镜像，再由 QEMU 启动后执行用户态测试。测试结果由 QEMU 原始日志、`results.json` 和 `metrics.csv` 同步保存，避免只根据终端上的一行“PASS”判断功能状态。

2.6.1 专项测试覆盖范围

表 2-3 调度专项测试与实际覆盖内容

测试程序	覆盖内容	实际状态
<code>prioritytest</code>	默认优先级、0 和 31 边界、非法参数、PID 查询与设置、 <code>nice</code> 、 <code>fork</code> 继承、进程槽位复用、同级与混合优先级调度。	单 CPU、3 CPU 均通过
<code>agingtest</code>	让 8 个高优先级 CPU-bound 进程持续竞争，观察低优先级进程是否在等待后逐级提升并获得运行机会。	单 CPU、3 CPU 均通过
<code>usertests -q</code>	检查进程退出、文件系统和原有 <code>xv6</code> 行为是否在调度修改后保持稳定。	单 CPU、3 CPU 均通过

2.6.2 QEMU 回归结果

本次验证使用 QEMU 11.1.1，在当前 main 快照的干净临时副本中构建，以避免本地工作区中与本项目无关的 README 删除改动。单 CPU 和 3 CPU 两组回归均完成了内核构建、文件系统镜像生成、QEMU 启动、用户态程序退出后的 shell 存活检查以及 usertests -q。核心结果如下：

表 2-4 单 CPU 与多 CPU 回归结果

配置	优先级专项	Aging 专项	其他回归
		AGING_RESULT:	
1 CPU	prioritytest: PASS, 输出 6 条指标	136 ticks 内从 3 提升至 0, 未超时	usertests -q: ALL TESTS PASSED
		AGING_RESULT:	
3 CPU	prioritytest -smp: PASS, 输出 12 条指 标	45 ticks 内从 3 提升至 0, 未超时	usertests -q: ALL TESTS PASSED

在 Aging 日志中，单 CPU 场景的高优先级竞争进程数为 8，high_alive=8，timeout=0；3 CPU 场景也得到相同的竞争进程数量与无超时结果。这里的 elapsed_ticks 只用于说明测试完成时机，不将一次本地回归误解为性能基准。测试脚本此前还暴露了在 macOS 上读取 Linux 环境文件的兼容问题，已增加通用后备路径，详见 docs/bugs.md。

第三章 问题分析与解决

3.1 QEMU 启动时动态库符号不匹配

项目早期使用 Homebrew 安装的 QEMU 时，程序因找不到 `cs_close` 符号而退出，机器上同时缺少 RISC-V 交叉编译工具链。该问题属于构建环境故障，与 xv6 内核代码无关。处理时安装 `riscv64-elf-gcc` 和 `riscv64-elf-binutils`，随后重装 QEMU 及其依赖，并在 Make 参数中使用 `riscv64-elf-` 工具链前缀。修复后 QEMU 能够引导 xv6 并进入 shell。

3.2 fork 继承优先级时缺少锁保护

最初的 `kfork()` 直接读取父进程的 `priority`。该字段会被系统调用修改，按 xv6 的进程字段约定应由 `p->lock` 保护。直接读取可能与并发写入发生竞争，也无法保证子进程获得一致快照。

修复后的代码先获取父进程锁，把优先级保存到局部变量后立即释放，再完成后续内存复制和子进程初始化。子进程继承该快照；若值非法则使用默认优先级 10。这样既遵守了字段的锁语义，也避免在耗时的地址空间复制过程中长期持有父进程锁。

3.3 xPack QEMU 版本号解析失败

Makefile 原有正则只识别以 `QEMU emulator version` 开头的输出。xPack QEMU 9.2.4 在前面增加了 `xPack`，导致版本字符串未被提取，后续传给 `bc` 时出现语法错误和整数判断异常。

修复时将开头的 `xPack` 设为可选部分，仍只提取主、次版本参与最低版本比较。改动没有放宽 QEMU 7.2 的最低要求：`xPack 9.2.4` 可以通过检查，旧版标准 QEMU 仍会被拒绝。

3.4 Aging 与手动调优的状态衔接

Aging 分支规定进程获得 CPU 后清零 `wait_ticks`，而系统调用分支最初没有统一手动调优后的等待状态。如果保留旧计数，刚被用户设置为新优先级的进程可能立即触发 Aging，手动设置的效果会混入此前的等待历史。

最终约定是：进程被调度运行时清零等待计数，`setpriority` 成功写入新优先级时也清零。两条路径都表示一次新的等待周期开始，语义简单，也便于说明和检查。该约定在模块合并时统一，并记录在 `docs/bugs.md` 中。

第四章 总结与展望

本项目在 xv6-RISC-V 原有调度框架内完成了四部分改造。进程控制块增加了优先级和等待计数字段，fork 可以安全继承父进程优先级。调度器按静态优先级选择可运行进程，并用轮转起点维持同级进程的基本公平。Aging 负责逐级提升持续等待的低优先级进程，系统调用和 nice 命令提供用户态查询与修改入口。实现沿用 xv6 的进程锁和状态转换，没有加入课题范围外的复杂调度结构。

选出优先级最高的进程只是调度修改的一部分。字段何时初始化和清零、fork 读取字段时是否持锁、多 CPU 下候选状态是否变化、Aging 如何衔接手动设置，都会影响内核运行。项目把这些约定落到少量字段、命名常量和明确的重置时机中，模块接口因而较容易核对。

第五阶段的专项测试程序已经合并到主分支，并在单 CPU、3 CPU 的 QEMU 环境中完成回归。prioritytest 覆盖了默认值、边界与非法参数、PID 查询与设置、nice、fork 继承、同级公平和不同优先级；agingtest 验证了高优先级竞争存在时低优先级进程能够逐级提升；两组配置下的 usertests -q 也均输出了 ALL TESTS PASSED。测试结果保存为原始日志与结构化结果文件，报告中的数值均来自本次实际 QEMU 执行。

本次集成还修复了测试运行器在 macOS 上读取 Linux 环境信息的问题，并在 docs/bugs.md 中记录了原因和验证方式。当前实现仍未把调度次数和运行时间作为新的持久化字段加入进程控制块；现阶段的调度统计由专项测试输出的 METRIC、AGING_RESULT 以及运行器汇总文件承担。后续若课程需要更细粒度的性能分析，可在不改变现有调度语义的前提下增加专门统计字段和长期实验设计。